

Midterm Project Report

AI Explanation Reliability Under Attack

github.com/Davidfaleye360/explanation-reliability-under-attack (private repository)

1. Project Overview

Title: AI Explanation Reliability Under Attack.

Problem. Image classifiers are often paired with explanation tools such as Grad-CAM, which draws a heatmap of the image regions the model relied on. Such heatmaps are sometimes treated as a sanity check on the model's answer. Adversarial attacks can change a classifier's prediction with tiny, hard-to-see changes to the image. It is not obvious whether the explanation also changes when that happens, or whether it stays almost the same while the answer becomes wrong. If it stays the same, an explanation-based check would give false reassurance.

Main question. How does adversarial attack strength affect the similarity between an image's Grad-CAM explanation before and after an attack, when the attack succeeds in changing the prediction?

Goals and objectives:

- Train a ResNet-18 classifier on CIFAR-10 to 85-93% test accuracy.
- Generate Grad-CAM explanations and confirm they focus on the objects in normal images.
- Implement FGSM and PGD attacks at three strengths each (epsilon 0.01, 0.03, 0.05), six configurations in total, and measure how often they succeed.
- Fix, before any attack explanation is examined, a similarity threshold derived from how much heatmaps change under harmless random noise.
- Compare clean and attacked heatmaps with SSIM (primary), IoU and correlation (supporting), and report how often an attacked explanation stays above the threshold.
- Test three hypotheses: H1, similarity falls as attack strength rises; H2, PGD and FGSM behave differently at comparable strength; H3, some successful attacks keep a highly similar explanation (a "deceptive" explanation).

Expected final outcome. A reproducible end-to-end pipeline in a GitHub repository, results tables and charts for all six configurations, a gallery of before/after examples, and written conclusions limited to this CIFAR-10 / ResNet-18 setting. The project measures explanation similarity. It does not measure whether a human finds an explanation plausible or whether Grad-CAM faithfully reflects the model's reasoning, and it includes no user study.

2. Work Completed

Work is organized by milestone, with dates taken from the repository's commit history (first commit September 4, latest October 6, 2026).

2.1 Project setup and planning (Sep 4)

- Created a private GitHub repository and the planned directory structure (src/, data/, results/, reports/, notebooks/), plus a .gitignore that keeps the dataset and model weights out of version control.
- Committed the project plan (PROJECT_PLAN.md): hypotheses, dataset, model, attack settings, metrics, the six-checkpoint schedule, and the rule that the deceptive-explanation threshold must be fixed before any attack result exists.

2.2 Data pipeline and classifier (Sep 4 - Sep 9)

- src/model.py: a torchvision ResNet-18 adapted for 32x32 images by replacing the 7x7 stride-2 stem with a 3x3 stride-1 convolution and removing the initial max-pool.
- src/train.py: loads CIFAR-10 through torchvision (downloaded from the University of Toronto host), applies RandomCrop(32, padding 4) and horizontal-flip augmentation to training images only, normalizes

with the dataset mean and standard deviation, trains with Adam and cross-entropy loss, logs train/test loss and accuracy each epoch, saves the best checkpoint, and plots training curves.

- Validation run (2 epochs): test accuracy rose from 57.3% to 70.4%. I then re-ran the whole pipeline myself in my own terminal, including a fresh 170 MB dataset download, and got 61.9% then 71.7%, confirming the pipeline reproduces.

2.3 Full training run (Sep 27)

- Trained for 30 epochs (Adam, learning rate 1e-3, batch size 128) on the Apple M2 GPU through PyTorch's MPS backend; epochs took 140-320 seconds each.
- Best test accuracy 91.03% at epoch 28, inside the 85-93% target. Training accuracy kept rising to about 97% while test accuracy levelled off near 90-91% after epoch 16, a mild, normal amount of overfitting. The epoch-28 checkpoint is used for everything downstream.

2.4 Grad-CAM integration (Sep 27)

- `src/gradcam_utils.py` wraps the `pytorch-grad-cam` library, targeting the last residual block (`layer4[-1]`), and later gained a batched version and an input-normalization helper.
- Generated heatmaps for one correctly classified test image per class (10 classes) and checked each by eye. In all ten, the strongest region sits on the object (for example the airplane's body, the cat's face, the ship's hull) rather than the background.

2.5 Noise-floor calibration and the locked threshold (Oct 6)

- `src/calibration.py` splits the 10,000 test images with a fixed random seed into 1,000 calibration images and 9,000 evaluation images that never overlap.
- Of the 1,000 calibration images, 905 are classified correctly and used. Each is paired with a copy carrying random ± 0.01 noise on every pixel channel (the same per-pixel size as an FGSM step at epsilon 0.01, but in a random direction).
- `src/metrics.py` computes SSIM between the clean and noisy Grad-CAM heatmaps for each pair. The 25th percentile of these 905 scores, 0.9802, is the fixed threshold. The script writes it once and refuses to overwrite it, and it was committed (1a45546) before any attack code existed.

2.6 Adversarial attacks (Oct 6)

- `src/attacks.py` builds FGSM (one step of size epsilon) and PGD (10 steps, step size $2.5 \times \text{epsilon} / 10$, random start, untargeted) with the `torchattacks` library at epsilon 0.01, 0.03 and 0.05.
- The classifier is wrapped so it accepts plain `[0, 1]` pixel images and normalizes internally, so epsilon has its intended pixel-scale meaning. A built-in check aborts the run if any perturbation exceeds its epsilon.
- Ran all six configurations on the 9,000 evaluation images. Every configuration produced thousands of successful attacks, far above the 500 the plan requires.

2.7 Documentation and reproducibility (Sep - Oct 6)

- Wrote a README documenting setup, run order, every attack, heatmap, SSIM and threshold parameter, results, and limitations.
- Saved the training logs, pinned exact library versions in `requirements.txt`, and added a `--seed` option and automatic log file to `train.py`.
- Applied feedback from earlier assignments: defined all attack and comparison parameters, described the outcome as explanation similarity, and added logs and reproducibility details.
- Wrote three progress reports (Foundation, Checkpoint 2, Attacks and Calibration).

2.8 Tools and technologies

Python 3.13.1; PyTorch 2.14.0 and torchvision 0.29.0 (model, data, MPS GPU); torchattacks 3.5.1 (FGSM, PGD); pytorch-grad-cam 1.5.7 (Grad-CAM); scikit-image 0.26.0 (SSIM); NumPy, pandas, matplotlib; Git and GitHub;

CIFAR-10 dataset; Apple M2 MacBook Air. I also used Claude Code, an AI coding assistant, to help write and debug code and draft documents under my direction.

3. Evidence of Progress

GitHub repository (private; access can be provided to the instructor): github.com/Davidfaleye360/explanation-reliability-under-attack. The repository holds all code, logs, tables, charts and reports described here. Commit history (17 commits, September 4 to October 6):

Date	Commit	Message
2026-09-04	2a81769	Initial commit
2026-09-04	76e6c13	Add project plan, repo scaffolding, and CIFAR ResNet-18 training script
2026-09-04	6241462	Remove assistant-facing notes from project plan
2026-09-04	f779765	Add --device override flag; default num_workers to 0
2026-09-04	06e1a7d	Add Checkpoint 1 validation training curve
2026-09-04	857a32b	Add Report 1 draft (Foundation checkpoint)
2026-09-09	e34edd2	Update Checkpoint 1 training curve with user-run results
2026-09-09	5c04aea	Update Report 1 with user-run validation results
2026-09-27	4027448	Add Grad-CAM wrapper targeting ResNet-18's final conv block
2026-09-27	1f0b366	Complete Checkpoint 2: full training run + Grad-CAM sanity gallery
2026-09-27	c643075	Switch gallery layout to a compact grid for easier report embedding
2026-09-27	82e6279	Add midterm report draft (Checkpoint 2)
2026-09-29	b4512df	Add PDF exports of Report 1 and Midterm Report
2026-10-06	1a45546	Lock deceptive-explanation threshold from noise-floor calibration
2026-10-06	aaf0ddc	Add FGSM/PGD attack evaluation, training logs, seeded logging, pinned deps
2026-10-06	7dbf768	Add attack results table and README with full experimental parameters
2026-10-06	1d1440b	Add Report 3 (Checkpoint 3: attacks and calibration baseline)

3.1 Classifier results

Run	Result
Validation run, 2 epochs (Sep 4)	Test accuracy 57.3% then 70.4%
Independent re-run, 2 epochs (Sep 9)	Test accuracy 61.9% then 71.7%
Full run, 30 epochs (Sep 27)	Best test accuracy 91.03% at epoch 28; final epoch 90.88%; final training accuracy 97.06%

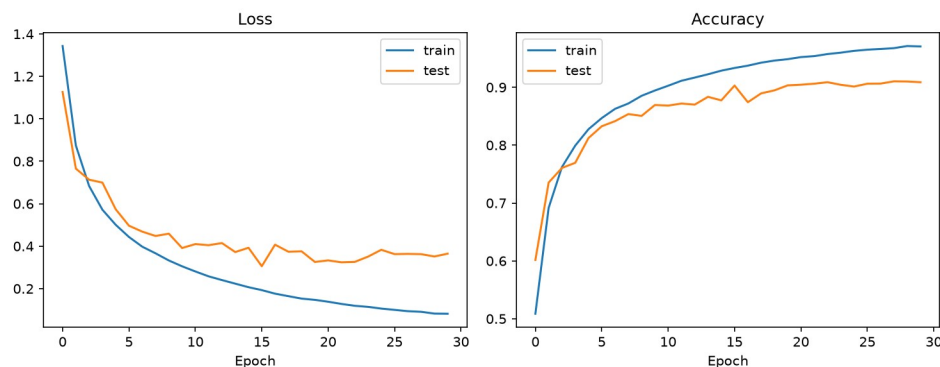


Figure 1. Training and test loss and accuracy over the 30-epoch run (log: results/logs/train_30epochs_2026-09-27.log).

3.2 Grad-CAM sanity check

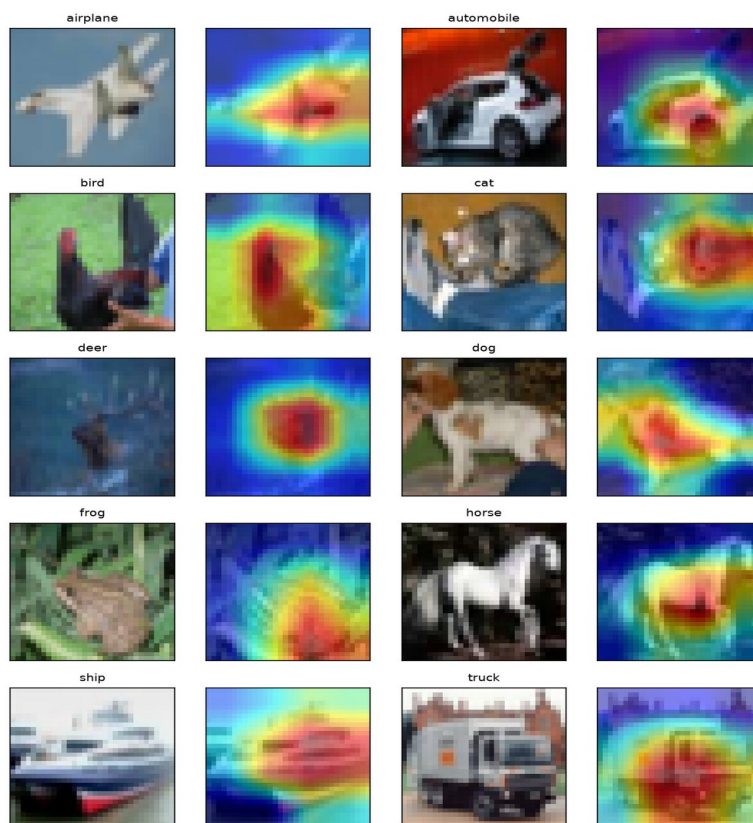


Figure 2. Original image and Grad-CAM overlay for one correctly classified test image per class.

3.3 Noise-floor calibration

Across the 905 clean/noisy pairs, SSIM between heatmaps has mean 0.969, median 0.994 and minimum -0.176. Random noise changed the predicted class for 13 of the 905 images. The locked threshold is the 25th percentile, SSIM 0.9802.

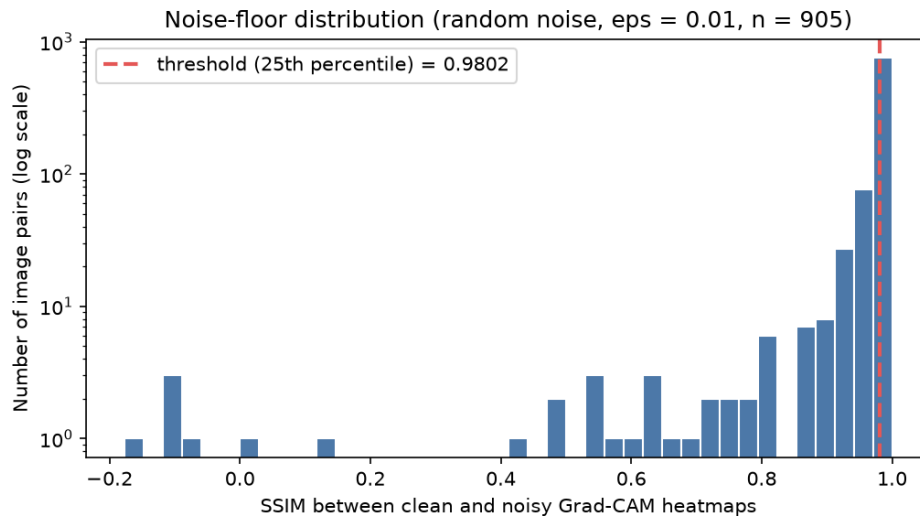


Figure 3. Noise-floor SSIM distribution (log scale) with the locked threshold.

3.4 Attack results

Evaluation split of 9,000 images; 8,198 were classified correctly before attack (91.09%). Success rate is the share of those that become misclassified.

Attack	Epsilon	Success rate	Adversarial accuracy	Images attacked
FGSM	0.01	77.6%	20.4%	6,362
FGSM	0.03	91.2%	8.0%	7,479
FGSM	0.05	91.9%	7.5%	7,531
PGD	0.01	97.7%	2.1%	8,011
PGD	0.03	100%	0.0%	8,198
PGD	0.05	100%	0.0%	8,198

PGD is stronger than FGSM at every epsilon. FGSM's success barely rises from epsilon 0.03 to 0.05. In every run the largest perturbation equalled its epsilon to floating-point precision (results/tables/attack_results.csv).

3.5 Code examples

Adapting ResNet-18 to 32x32 images (src/model.py):

```
model = resnet18(weights=None, num_classes=num_classes)
model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)
model.maxpool = nn.Identity()
```

Random calibration noise, same per-pixel size as an FGSM step (src/calibration.py):

```
signs = torch.randint(0, 2, images_01.shape, generator=generator).float() * 2 - 1
return (images_01 + eps * signs).clamp(0, 1)
```

Guard that stops any attack that exceeds its budget (src/attacks.py):

```
if max_linf > eps + 1e-5:
    raise RuntimeError(f"Attack exceeded its budget: max |change| = {max_linf:.5f}")
> eps = {eps}"
```

4. Progress Compared with the Original Plan

I compared progress against the checkpoint schedule in my project plan (PROJECT_PLAN.md).

Planned task	Status	Notes
Repo structure, README, plan committed	Done	
ResNet-18 for CIFAR-10 and training script	Done	Checkpoint 1
Short validation training run	Done	Reproduced independently
Full training to 85-93% accuracy	Done	91.03%
Grad-CAM wrapper and 10 sanity heatmaps	Done	All 10 classes look sensible
FGSM and PGD at three epsilons, six configurations	Done	Success rate and adversarial accuracy recorded
Noise-floor calibration and locked threshold	Done	SSIM $\geq$ 0.9802
SSIM metric	Done	Done early: needed for calibration
IoU and correlation metrics	Not started	Checkpoint 4
Attack, explain, compare pipeline	Not started	Checkpoint 4
Full-scale run, analysis, charts, hypotheses H1-H3	Not started	Checkpoint 5
Final report, polished repo, demo video	Not started	Checkpoint 6

Schedule. Checkpoints 1, 2 and 3 of the six are complete. The plan maps the midterm to Checkpoint 2, so the project is one checkpoint ahead of that mapping. Everything the plan assigns to Checkpoints 1-3 is finished.

Changes to the original plan, and why:

- The calibration set is held out of the test set (1,000 images, seeded split), and attacks are evaluated on the remaining 9,000. The plan said "held-out" without a source; this keeps the threshold from touching the images it later judges.
- The calibration noise is random ± 0.01 sign noise. The plan said small random noise at the smallest epsilon; sign noise matches an FGSM step's per-pixel size.
- PGD settings (10 steps, step size $2.5 \times \text{epsilon} / 10$, random start) were chosen explicitly because the plan did not specify them.
- Calibration was run and committed before the attack code was written, to follow the plan's rule literally. SSIM was implemented in Checkpoint 3 instead of 4 because calibration needs it.
- Added a README, saved training logs, pinned library versions and a seed option, in response to feedback about reproducibility.

5. Challenges and Solutions

Challenge 1: training that hung for hours.

The first training run took 2 hours 40 minutes for one epoch while using only about 52 seconds of processor time, and even import torch sometimes hung. This blocked all progress for a stretch. I ruled out the data-loader workers and the GPU backend, then found that the project folder sat in an iCloud-synced Desktop folder and that macOS sync daemons were contending for file access. Moving the project and virtual environment to a normal folder cut the import time from hours to about one second. Resolved.

Challenge 2: attacks exceeding their allowed perturbation.

A first smoke test of the attacks showed per-pixel changes about four times larger than epsilon. This would have made every attack success rate invalid. The cause was a torchattacks normalization option I had used incorrectly. I wrapped the model so it takes $[0, 1]$ pixels, discarded the invalid numbers, and added a check that stops any run whose perturbation exceeds epsilon. In the final run every perturbation matched its epsilon. Resolved.

Challenge 3: a very slow dataset download.

The 170 MB CIFAR-10 download ran at roughly 140 kB/s and took 19 to 25 minutes each time it was needed. The dataset is cached in data/ so it downloads once per machine. Resolved.

Challenge 4: reproducibility gaps.

Feedback on an earlier report noted missing training logs and setup details. I saved both training logs, pinned exact library versions, documented every parameter, and added a seed option. The existing 30-epoch checkpoint came from an unseeded run, so retraining will not match it exactly; this is recorded in the log. Partly resolved.

Open risk: a strict threshold.

Random noise barely changes the heatmaps, so the locked threshold (0.9802) is high and an attacked explanation must be nearly identical to count as deceptive. Grad-CAM at this layer is only 4x4 cells upsampled to 32x32, which also makes heatmaps smooth. The rule must not be changed after seeing results, so a low deceptive rate would be reported as a result. I will also report the full similarity distributions, not only the thresholded rate. Open.

6. Current Project Status

At the midpoint the foundation of the experiment is built and validated, and the rule for judging results is locked. Major outcomes so far:

- A ResNet-18 classifier with 91.03% test accuracy, with logs and a reproducible training script.
- A working Grad-CAM implementation whose heatmaps focus on the objects in all 10 classes.

- Six working attack configurations with measured success rates (FGSM 77.6% to 91.9%, PGD 97.7% to 100%).
- A locked deceptive-explanation threshold (SSIM ≥ 0.9802) from a 905-pair noise-floor calibration, committed before any attack explanation was examined.
- A documented, version-pinned repository with a README and three progress reports.

Not yet done: comparing clean and attacked explanations (the core measurement), the IoU and correlation metrics, the full-scale run, the charts, testing hypotheses H1-H3, and the final report and video. No conclusion about any hypothesis has been drawn yet. The first and only observation about explanation similarity so far is the noise-floor result above.

7. Plan for the Second Half of the Semester

Checkpoint 4: comparison pipeline on a subset

- Add IoU (top 20% of heatmap pixels) and correlation to `src/metrics.py` as supporting metrics.
- Build `src/pipeline.py`: for each successfully attacked image, compute clean and attacked Grad-CAM heatmaps, their SSIM, IoU and correlation, and apply the fixed 0.9802 threshold.
- Run it on 500 to 1,000 images per configuration and save side-by-side before/after examples, including any deceptive cases.

Checkpoint 5: full-scale results and analysis

- Run the pipeline on all successfully attacked evaluation images for all six configurations.
- Write `src/analyze.py` to produce per-configuration tables and charts of SSIM and deceptive rate against epsilon, split by FGSM and PGD.
- Evaluate H1, H2 and H3 against the data, including the case where there is no clear trend.

Checkpoint 6: final report and presentation

- Finalize conclusions limited to the CIFAR-10 and ResNet-18 setting, curate the strongest before/after examples, clean and document the repository, and record the final demo video.

Main risks: a low deceptive rate because of the strict threshold, and coarse 4x4 heatmaps. Both will be reported honestly with the full similarity distributions.